What are software requirements? How to analyse the requirements?

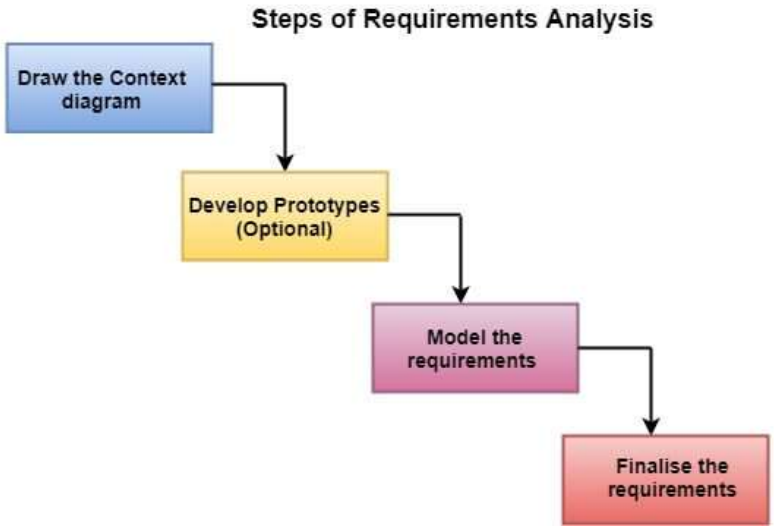
Software requirements are a crucial part of software development that define what a software system should accomplish. They are a detailed description of the functionality, features, constraints, and quality attributes that a software application must possess to meet the needs of its users or stakeholders. Requirements serve as the foundation for the design, development, and testing of a software system.

Here's a general overview of how to analyze software requirements:

1. Requirements Elicitation: This is the first step in gathering requirements. It involves identifying and engaging with stakeholders (such as clients, users, managers, and developers) to understand their needs and expectations. Various techniques like interviews, surveys, workshops, and observations can be used to extract requirements.
2. Requirements Documentation: Document the gathered requirements in a clear, structured, and organized manner. Common documentation formats include use cases, user stories, requirement documents, and diagrams (e.g., UML diagrams).
   Requirements Analysis:
3. Functional Requirements: Identify what the software should do. Break down high-level functions into detailed functionalities, features, and operations.
4. Non-Functional Requirements: These specify quality attributes like performance, security, reliability, and usability. Analyze these to understand the system's overall behavior.
5. Constraints: Identify any constraints or limitations that the software must adhere to, such as budget, technology choices, or legal requirements.
   Prioritization and Validation:
6. Prioritize requirements based on their importance and urgency. This helps in making decisions about what to include in the software's initial release and what can be deferred.
7. Validate requirements by ensuring they are clear, complete, and consistent. This may involve reviewing them with stakeholders to confirm understanding and agreement.
8. Requirement Traceability: Establish traceability between requirements, design elements, and test cases. This ensures that every requirement is addressed during development and testing, helping in comprehensive coverage.
9. Change Management: Be prepared for changes in requirements as the project progresses. Use a change management process to assess the impact of changes on the project's scope, schedule, and budget.
10. Prototyping and Modeling: Create prototypes or models to visualize the software's behavior and user interfaces. These can help stakeholders better understand the requirements and provide feedback.
11. Review and Validation: Review the requirements with stakeholders to ensure they accurately reflect the desired system. Validation involves confirming that the software, once developed, meets these requirements.
12. Documentation and Communication: Maintain up-to-date documentation of requirements throughout the project lifecycle. Effective communication is essential to keep all stakeholders informed about changes and progress.

13. Continuous Monitoring and Feedback: As the project progresses, continuously monitor and update the requirements based on feedback and evolving project needs. Agile methodologies, in particular, emphasize iterative development and adaptive requirements management.

Requirement analysis is significant and essential activity after elicitation. We analyze, refine, and scrutinize the gathered requirements to make consistent and unambiguous requirements. This activity reviews all requirements and may provide a graphical view of the entire system. After the completion of the analysis, it is expected that the understandability of the project may improve significantly. Here, we may also use the interaction with the customer to clarify points of confusion and to understand which requirements are more important than others.



What is software? What are the development lifecycle phases?

Software refers to a collection of computer programs, data, and related instructions that enable a computer system to perform specific tasks or functions. It includes both the code written by software developers and the data that the software operates on. Software can take various forms, including applications, operating systems, libraries, and more. It is a crucial component of modern computing and technology, enabling computers to perform a wide range of tasks, from word processing and web browsing to complex scientific simulations and data analysis.

The Software Development Lifecycle (SDLC) is a structured process that defines the stages or phases involved in developing software. These phases help manage the development process, ensure quality, and facilitate communication among project stakeholders. While there are various SDLC models, a commonly used one is the Waterfall model, which consists of several sequential phases:

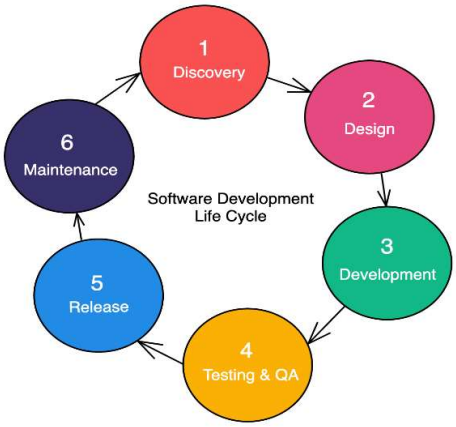
1. Requirements Gathering: In this phase, the project team works closely with stakeholders to gather and document the software's functional and non-functional requirements.
2. System Design: Based on the requirements, the system architecture and design are created. This phase involves defining the overall structure of the software, including data models, user interfaces, and system components.

3. Implementation (Coding): Developers write the actual code for the software based on the design specifications. This phase involves programming, testing individual modules, and integrating them into the overall system.
4. Testing: The software is rigorously tested to identify and fix defects or bugs. This includes unit testing, integration testing, system testing, and user acceptance testing (UAT).
5. Deployment (or Installation): The software is deployed to production environments or distributed to end-users. Installation procedures and documentation are prepared.
6. Maintenance and Support: After deployment, ongoing maintenance and support are provided to address issues, bugs, and updates as needed. This phase can continue for the lifetime of the software.

While the Waterfall model is a linear and sequential approach, other SDLC models, like the Agile model, promote iterative and incremental development. In Agile, the development process is broken into smaller cycles or iterations, with each iteration delivering a functional increment of the software. Agile allows for more flexibility and adaptation to changing requirements.

Other SDLC models include the V-Model, Spiral Model, and Lean Software Development, each with its own variations and strengths, depending on the project's nature and requirements.

In addition to these core phases, software development often involves activities like project planning, documentation, version control, and continuous integration and deployment (CI/CD) to ensure the successful creation and maintenance of software systems. The choice of SDLC model and specific phases used can vary depending on the project's goals, complexity, and organizational preferences.



Explain briefly requirements elicitation.

Requirements elicitation is the process of identifying, gathering, and documenting the requirements for a software system. It is a crucial early phase in the software development lifecycle (SDLC) because it sets the foundation for what the software will ultimately do and how it will fulfill the needs of its users and stakeholders.

Here's a brief explanation of the requirements elicitation process:

1. Identifying Stakeholders: The first step is to identify and engage with all relevant stakeholders. Stakeholders are individuals or groups who have an interest in the

software project, such as clients, end-users, business analysts, project managers, and developers.

2. Stakeholder Communication: Establish clear channels of communication with stakeholders. This may involve conducting interviews, surveys, workshops, meetings, or even informal discussions to understand their perspectives, expectations, and requirements.
3. Gathering Requirements: Actively collect information from stakeholders about what the software needs to accomplish. This may involve asking open-ended questions, seeking examples, and encouraging stakeholders to express their needs and desires.
4. Analyzing Requirements: As requirements are gathered, they should be analyzed to ensure they are clear, complete, and unambiguous. It's important to identify any conflicts or inconsistencies between different stakeholders' requirements.
5. Documenting Requirements: Record the requirements in a structured and organized manner. Common formats for documenting requirements include use cases, user stories, requirement documents, and diagrams like flowcharts or UML diagrams.
6. Prioritization: Not all requirements are of equal importance. Work with stakeholders to prioritize requirements based on their criticality, impact, and urgency. This helps in making decisions about what to implement first.
7. Validation: Validate the requirements by reviewing them with stakeholders to ensure that they accurately represent their needs and expectations. This is an iterative process that helps refine and clarify requirements.
8. Change Management: Be prepared for changes in requirements as the project progresses. Implement a change management process to assess the impact of changes on project scope, schedule, and budget.
9. Traceability: Establish traceability between requirements, design elements, and test cases. This ensures that each requirement is addressed during development and testing.
10. Documentation and Communication: Maintain clear and up-to-date documentation of requirements throughout the project. Effective communication is essential to keep stakeholders informed about changes and progress.

Requirements elicitation is a collaborative and iterative process that requires effective communication and engagement with stakeholders. It helps ensure that the software development project starts with a clear understanding of what needs to be achieved, which is essential for delivering a successful software product that meets the needs of its users and stakeholders.

What is agile development? Explain two perspectives on scaling of agile methods.

Agile development is an iterative and incremental approach to software development that emphasizes flexibility, collaboration, and customer-centricity. It was conceived as a response to the limitations of traditional, plan-driven development methodologies, such as

the Waterfall model. Agile methods prioritize delivering functional software in smaller, manageable increments, which allows for more frequent adaptation to changing requirements and customer feedback.

Two common perspectives on scaling Agile methods to larger, more complex projects or organizations are:

1. Scaled Agile Framework (SAFe):

   Description: SAFe is one of the most widely adopted frameworks for scaling Agile. It provides a comprehensive approach to scaling Agile practices from teams to large organizations. SAFe builds on Agile principles and extends them to larger teams and complex projects.

   Key Features:

   Roles and Responsibilities: SAFe introduces roles like Release Train Engineer (RTE), Product Owner, and Scrum Master at various levels to coordinate activities and facilitate communication.

   Program Increment (PI): SAFe organizes development into fixed-duration Program Increments (typically 8-12 weeks) where a set of features is developed, integrated, and demonstrated.

   Value Stream: It emphasizes defining and managing value streams, which represent the sequence of activities required to deliver value to customers.

   Cadence and Synchronization: Teams synchronize their work through common ceremonies such as PI planning, Inspect and Adapt (I&A) workshops, and Scrum-of-Scrums.

   Portfolio Management: SAFe includes mechanisms for aligning and prioritizing work across multiple Agile Release Trains (ARTs) to ensure that the most valuable initiatives are pursued.

2. Large Scale Scrum (LeSS):

   Description: LeSS is a minimalist approach to scaling Agile that extends Scrum principles and practices. It focuses on simplicity and removing unnecessary roles and artifacts while preserving the core Scrum framework.

   Key Features:

   Scrum at Scale: LeSS promotes the idea that Scrum can be scaled without adding complex layers of roles or processes. It recommends having one Product Owner and one Product Backlog for the entire product.

   Feature Teams: LeSS encourages the use of feature teams (cross-functional teams) to work on all aspects of a product rather than component-based teams.

Sprint Planning: LeSS retains Sprint planning ceremonies but may involve more teams in coordination during these planning sessions.

Transparency and Empiricism: LeSS relies on the same core Scrum principles of transparency, inspection, and adaptation but applies them at scale.

Continuous Improvement: LeSS emphasizes continuous improvement and learning, encouraging teams to experiment with their processes and practices.

Differentiate between Waterfall model and RAD model.

Waterfall Model:

1. Phases: The Waterfall model follows a sequential and linear approach, where each phase is completed before moving on to the next. The typical phases include requirements, design, implementation, testing, deployment, and maintenance.
2. Requirements: In the Waterfall model, all requirements are gathered and documented at the beginning of the project. Changes to requirements are generally discouraged after the project has started.
3. Flexibility: It is inflexible when it comes to changing requirements or scope. Any changes in requirements often require going back to the initial phases, which can be time-consuming and costly.
4. Testing: Testing is typically performed at the end of the development process, after coding is complete. This means that defects may not be discovered until late in the project lifecycle.
5. Client Involvement: Client involvement is primarily at the beginning and the end of the project, with limited opportunities for feedback and changes during development.
6. Suitability: It is best suited for projects with well-defined, stable requirements and minimal expected changes.

RAD Model (Rapid Application Development):

1. Phases: The RAD model is iterative and focuses on rapid prototyping and development. It consists of four primary phases: Requirements Planning, User Design, Rapid Construction, and Cutover.
2. Requirements: In RAD, only the high-level requirements are gathered initially. Detailed requirements are defined and refined as the project progresses through iterative cycles.
3. Flexibility: RAD is highly flexible and adaptable to changing requirements. It allows for continuous refinement and adaptation based on user feedback.
4. Testing: Testing is integrated throughout the development process. As each iteration produces a partial system, testing can begin early, which helps in early identification and resolution of issues.

5. Client Involvement: RAD emphasizes continuous client involvement and feedback. Clients are actively engaged throughout the development process, which ensures that the final product meets their needs.
6. Suitability: RAD is suitable for projects where requirements are not well understood or are likely to change. It is particularly effective for projects where user involvement and feedback are critical.

Functional and non-functional requirement.

Functional Requirements:

1. What the System Should Do: Functional requirements describe the specific functions, features, and capabilities that a software system must have. They define the system's behavior and outline what it should do when specific inputs or conditions are met.
2. User Interactions: These requirements focus on user interactions with the system, such as user interfaces, user workflows, and how users will interact with various features and functions.
3. Specific Actions: Functional requirements are often written in the form of use cases or user stories, which outline specific actions that users or system components can perform and the expected outcomes.
4. Testable: Functional requirements are typically testable because they specify specific behaviors or outcomes that can be verified through testing. Test cases can be developed to confirm that the system meets these requirements.

Non-Functional Requirements:

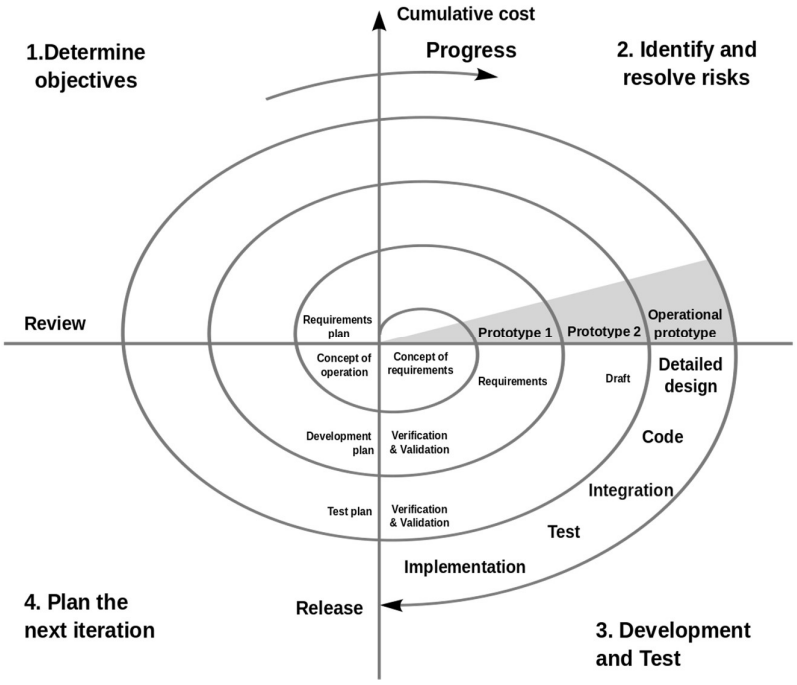
1. How the System Should Perform: Non-functional requirements define the qualities or attributes that describe how the system should perform. They are concerned with aspects other than specific features and functions, focusing on system-wide characteristics.
2. Quality Attributes: These requirements address quality attributes or characteristics such as performance, scalability, security, reliability, usability, and maintainability.
3. Constraints: Non-functional requirements may also include constraints, such as budget limitations, regulatory compliance, or technology choices.
4. Often Harder to Test: Non-functional requirements can be more challenging to test directly because they relate to overall system behavior rather than specific actions. Performance testing, security testing, and other specialized tests may be needed.

Draw and explain Spiral model.

The Spiral model is a risk-driven software development process model that combines elements of iterative development with aspects of the Waterfall model. It was developed by Barry Boehm in 1986 and is particularly well-suited for large, complex projects where risks and uncertainties are high. The model is called "Spiral" because it involves a series of cycles or spirals, each representing a phase in the software development process.

Here's an explanation of the Spiral model:

1. Phases:
   - Planning: In the first phase, the project is defined, and objectives, constraints, and alternative solutions are identified. Risks are also assessed during this phase.
   - Risk Analysis: This phase involves a detailed risk analysis, where potential risks and uncertainties are identified and analyzed. Risk mitigation strategies are developed to address these risks.
   - Engineering: The actual development and testing of the software occur in this phase. This phase follows an iterative and incremental approach, with small portions of the software being developed in each spiral.
   - Evaluation: After each cycle, the developed software is evaluated. This evaluation may involve user feedback, testing, and performance analysis.
2. Spiral Cycles: Each cycle in the Spiral model represents a complete iteration through these phases. The number of cycles can vary based on the project's complexity and the level of risk involved. The Spiral model allows for multiple iterations of these cycles.
3. Risk-Driven: The key feature of the Spiral model is its focus on risk management. Risks are identified and addressed throughout the development process. If significant risks are identified, the project may return to earlier phases to modify the plan or develop alternative solutions.
4. Flexibility: The Spiral model allows for flexibility in terms of making changes to the software as the project progresses. This is particularly valuable when requirements are not well-understood or may change.
5. Progressive Refinement: Each cycle in the Spiral model results in a more refined and improved version of the software. This progressive refinement continues until the software is ready for deployment.
6. Documentation: Documentation is maintained at each phase to capture the project's objectives, risks, and progress. This documentation is crucial for making informed decisions throughout the development process.
7. Closure: The project is closed when the software is deemed complete and ready for deployment. Maintenance and support may follow, depending on the project's requirements.



Explain waterfall and prototyping process model with its merits and demerits.

Waterfall Model:

Description: The Waterfall model is a traditional and sequential software development process model. It divides the development process into discrete phases, and each phase must be completed before moving to the next one. The phases typically include requirements, design, implementation, testing, deployment, and maintenance.

Merits:

1. Clarity and Structure: The Waterfall model provides a structured and well-defined approach to development, making it easier to manage and understand.
2. Documentation: Each phase produces specific documentation, which is valuable for tracking progress and maintaining a clear project history.
3. Quality Assurance: Testing is a dedicated phase in Waterfall, allowing for thorough testing and quality assurance before deployment.

Demerits:

1. Inflexibility: The model is inflexible when it comes to changes in requirements or scope. Any changes often require going back to earlier phases, which can be time-consuming and costly.

2. Late User Feedback: User feedback is typically obtained after development is complete, which can lead to misunderstandings and dissatisfaction if the end product does not meet user expectations.
3. Long Delivery Time: Since the software is developed in a linear fashion, users may have to wait a long time before they can use the final product.

Prototyping Model:

Description: The Prototyping model is an iterative development process where a simplified version of the software, called a prototype, is built early in the project to gather user feedback and refine requirements. It involves creating a working model of the software to help clarify requirements and design.

Merits:

1. User Involvement: Prototyping encourages active user involvement and feedback from the early stages, leading to better alignment with user needs and expectations.
2. Early Visualization: Stakeholders can visualize the software early in the process, which helps in refining requirements and design.
3. Flexibility: The model is highly adaptable to changes and evolving requirements, making it suitable for projects with uncertain or rapidly changing needs.

Demerits:

1. Incomplete System: Prototypes may not represent the full functionality of the final system, leading to misunderstandings about what the final product will include.
2. Scope Creep: Frequent changes and additions to the prototype can lead to scope creep, where the project expands beyond its initial scope.
3. Potential for Misinterpretation: Users might focus on the prototype's appearance and not understand that it's a work in progress, leading to misconceptions about the project's status.
4. Documentation Challenges: Managing and documenting changes in an evolving prototype can be challenging, potentially leading to incomplete or outdated documentation.

Draw and Explain different process activities.

1. Requirements Gathering and Analysis:

Description: This activity involves understanding and documenting the needs and expectations of stakeholders. It includes gathering, analyzing, and formalizing requirements for the software system.

Key Steps: Interviews, surveys, workshops, and documentation analysis are used to collect requirements. The goal is to ensure a clear understanding of what the software should do.

2. System Design:

Description: In this activity, the high-level architecture and design of the software system are created based on the gathered requirements. It involves defining the system's structure, components, and data flow.

Key Steps: Design diagrams (e.g., UML diagrams), data models, and system architecture documents are created to represent the system's design.

3. Implementation (Coding):

Description: This activity involves writing the actual code for the software based on the design specifications. It includes programming, coding reviews, and unit testing of individual modules.

Key Steps: Developers write code following coding standards and guidelines. The code is tested at the unit level to identify and fix defects.

4. Testing:

Description: Testing activities involve systematically evaluating the software to identify and fix defects, validate functionality, and ensure it meets the specified requirements.

Key Steps: Various testing phases include unit testing, integration testing, system testing, user acceptance testing (UAT), and regression testing.

5. Deployment:

Description: Deployment activities involve installing the software in the target environment, configuring it, and making it available to users.

Key Steps: Deployment plans and procedures are executed, and any necessary data migration or system setup is performed.

6. Maintenance and Support:

Description: After deployment, the software enters the maintenance phase. This activity involves ongoing support, bug fixes, updates, and enhancements to ensure the software's continued functionality.

Key Steps: Teams monitor the software's performance, address reported issues, and implement updates or enhancements based on user feedback and evolving requirements.

7. Documentation:

Description: Documentation activities include creating and maintaining project documentation, such as requirements documents, design specifications, user manuals, and technical documentation.

Key Steps: Documenting each phase of the software development process is essential to ensure clear communication, knowledge transfer, and future reference.

8. Quality Assurance (QA):

Description: QA activities are integrated into various phases of the development process to ensure that the software meets quality standards and adheres to best practices.

Key Steps: QA involves reviewing code, conducting inspections, performing testing, and implementing quality control measures.